

TesseraSeal

A Post-Quantum Encryption Envelope

with optional X25519 hybrid (suite `0x06`)

Bahram Samani

`tessera` monorepo maintainer

Whitepaper, Version 1.3

Released 2026-05-21 · revised 2026-06-10 (truthfulness errata, TSEAL-00045)

Build date: June 16, 2026 **Build time:** 00:01 UTC
Source: `tessera-seal/latex/main.tex`
Spec version: v1.3 **Wire variant:** `0x04` **Suites:** `0x01`, `0x02`, `0x06`

This document is the canonical reference for TesseraSeal v1.x. It supersedes the v1 (2026-05-20) and v1.2 (2026-05-21 Python-only hybrid) drafts. See section 15, Appendix F, for the v1.0–v1.3 release timeline.

Contents

Conventions **3**

TL;DR **3**

1 Why TesseraSeal **4**

2 Threat Model **4**

3 Wire Format (Normative) **5**

4 Algorithm Suites **6**

5 ML-KEM-1024 Binding **7**
5.1 Tight QROM proof . . . 7
5.2 Implementation sources (v1.x, as shipped) . . . 8
5.3 The "Kyber" naming ban . . . 8

6 Hybrid Suite `0x06` (Optional) **9**
6.1 Construction . . . 9
6.2 Sizes . . . 9
6.3 Security argument . . . 9
6.4 Cost . . . 10
6.5 Constant-time and supply-chain posture . . . 10

7 AEAD and Commitment **10**
7.1 Why a separate commitment tag . . . 10
7.2 The UtC construction (Bellare & Hoang 2022) . . . 11
7.3 Verification . . . 11
7.4 Why UtC over CTX, GCM-SST, or compactly-committing schemes . . . 11
7.5 Commitment vs content binding . . . 11

8 Key Derivation and Deterministic Nonce **12**
8.1 Key schedule . . . 12
8.2 Salt binding . . . 12
8.3 Why deterministic nonce . . . 12
8.4 Multi-user / multi-key bounds . . . 13

9 The TesseraID-as-AD Canonical Pattern **13**

10 Failure Modes **14**
10.1 One error class for every authentication-class failure . . . 14
10.2 Constant-time discipline . . . 14
10.3 Timing measurement (automated script; not yet a blocking gate) . . . 15
10.4 Implicit-reject semantics . . . 15

11 Side Channels **15**

11.1 Known attacks 2024–2026 . . . 15
11.2 What v1 mitigates . . . 16
11.3 What v1 does *not* mitigate . . . 16
11.4 Forward path . . . 17

12 Plugin Protocol **17**
12.1 Why a plugin protocol in v1 . . . 17
12.2 Wire format . . . 17
12.3 Reference plugin: `tessera-seal-plugin-file` . . . 17
12.4 What the protocol does *not* cover . . . 18

13 Conformance **19**
13.1 KAT vector files . . . 19
13.2 Cross-language runner . . . 19
13.3 Hybrid suite `0x06` skip discipline . . . 20
13.4 Local invocation . . . 20
13.5 CI gate . . . 21
13.6 Vector generation and the append-only freeze . . . 21

14 Comparison with Existing Formats **21**
14.1 Envelope formats . . . 22
14.2 KEM cost (Zen 4, cycles; sizes in bytes) . . . 23
14.3 AEAD throughput (1 MiB, single thread, GB/s) . . . 23
14.4 Reference benchmarks (Apple M4 Max, ops/sec, v1.3) . . . 23
14.5 Deployment scale (mid-2026) . . . 24

15 Appendices **24**

Conventions

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals, as shown here.

TL;DR

TesseraSeal is a 16-byte-header, variable-body **post-quantum encryption envelope** format with byte-identical polyglot reference implementations under the `tessera` monorepo. It joins TesseraID (41-byte identifier, variant bytes `0x01`/`0x02`) and TesseraHash (73-byte content hash, variant byte `0x03`) as the third project in the family, owning variant byte `0x04`.

v1.x ships three suites, allocated at the `SUITE` byte (header offset 7):

- **Suite `0x01` (mandatory, pure PQ). ML-KEM-1024** (fip, 2024) + **AES-256-GCM** + a **32-byte UtC commitment tag** (Bellare and Hoang, 2022) + **HKDF-SHA3-256**. The v1 baseline.
- **Suite `0x02` (optional, pure PQ).** Substitutes ChaCha20-Poly1305 for callers without AES hardware. Same UtC commitment.
- **Suite `0x06` (optional, hybrid; v1.2).** Adds **X25519** (Langley et al., 2016) alongside ML-KEM-1024 with an X-Wing–style SHAKE-256 combiner (Connolly et al., 2025). Opt-in defense-in-depth for callers who want a classical safety net against a Module-LWE structural break.

Suites `0x01`/`0x02` carry the unambiguous post-quantum confidentiality promise: the residual Module-LWE-break risk (sections 2 and 15) is the deliberate cost. Suite `0x06` is the *opt-in* alternative that mitigates that residual risk; it is a separate suite-byte allocation, never an in-place modification of `0x01`/`0x02`. The wire is unambiguous about which promise a given envelope makes.

At a glance

- **Three suites, three explicit promises.** No agility within a suite, no negotiation, no downgrade. A suite byte *is* a version.
- **Key-committing.** Every envelope carries a 32-byte UtC commitment tag (HMAC-SHA3-256) — closes the multi-recipient confusion class that age v1.x still has open.
- **Deterministic AEAD nonce** derived from a per-envelope-fresh session key. No CSPRNG dependency on the encrypt path beyond ML-KEM's internal `Encaps` randomness (and the X25519 ephemeral keygen in hybrid).
- **Single `DecryptionError`** for every authentication-class failure. No distinguishing oracle. Structural errors (truncation, unknown variant/suite, suite/key mismatch) are distinguishable safely.
- **Binds TesseraID as AD** by convention. Envelope rebinding fails closed (section 9).
- **Plugin protocol ships in v1.** Key-on-disk lives in the plugin, not the core spec. Reference `tessera-seal-plugin-file` uses `.tseal-key-v1.1` (Argon2id (rfc, 2021) +

AES-256-GCM key wrap with embedded public key).

- **Inherits monorepo discipline.** Spec-first LaTeX (this document); Python is the readable reference; C pins the canonical ABI; cross-language KAT vectors gate CI.
- **Polyglot status (2026-06-13):** all eight ports ship suites `0x01`/`0x02`. Hybrid `0x06` is a 4-way harness-enforced byte-identical KAT contract across Python, Rust, Go, and C (TSEAL-00030), and additionally ships at the library level in C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034) — each proven by in-suite `hybrid_*.json` KAT byte-identity even without a TesseraSeal conformance runner. Java (TSEAL-00033) is the lone remaining hybrid gap (section 13, Appendix F).

1 Why TesseraSeal

Two forces are reshaping how large systems handle data, and they pull in the same direction.

Scale-out with no coordinator. Distributed systems — and machine-learning training clusters in particular, now pushing past one million accelerators — mint identifiers and seal artifacts on every node with no central allocator in the hot path. The TesseraID sibling format in the tessera monorepo answers the identity half with a 256-bit random tail (the birthday bound keeps collisions near 10^{-41} even across 10^{18} identifiers) while staying time-sortable via a 48-bit millisecond prefix.

Data that outlives its cryptography. Training corpora, checkpoints, ledgers, and provenance are retained for years — across the expected arrival of cryptographically-relevant quantum computers. This is where TesseraSeal earns its place. It encrypts artifacts under ML-KEM-1024 (FIPS 203) with a key-committing AEAD, so *harvest-now, decrypt-later* fails against anything sealed today, and a single envelope cannot be made to open to two different plaintexts under two different keys — the multi-recipient confusion that matters when thousands of nodes consume the same sealed dataset. TesseraSeal interlocks with its siblings: bind a TesseraID as the AEAD associated data and the envelope is cryptographically pinned to the record it belongs to; bind a TesseraHash of the plaintext and it is pinned to specific content. TesseraID (256-bit tail) and TesseraHash (512-bit SHAKE-256 tail) are sized for the same post-quantum horizon, so the identifier, the content address, and the ciphertext all remain hard targets together.

2 Threat Model

The attacker is computationally bounded by the IND-CCA2 security of ML-KEM-1024 (fip, 2024; Hofheinz et al., 2017) and the confidentiality and integrity of the symmetric AEAD (AES-256-GCM under its standard assumptions, augmented with the UtC commitment tag (Bellare and Hoang, 2022)). They see every byte of every envelope, may control delivery (replay, reorder, drop), may have a decryption oracle for *other* ciphertexts under the same recipient key (IND-CCA2 required), and may flip any header byte or substitute associated data (the AEAD's AD path authenticates both).

The attacker **cannot** compromise the recipient's long-term ML-KEM-1024 private key (and, for hybrid suite `0x06`, the X25519 long-term scalar): that is the plugin protocol's responsibility (section 12). The attacker may attempt *multi-recipient/AEAD-key confusion* (Grubbs et al., 2017;

Albertini et al., 2020; Len et al., 2021); the 32-byte UtC commitment tag closes this class on every suite, including hybrid.

The attacker may attempt *side-channel attacks* against ML-KEM: KyberSlash 1/2 timing leaks (Bernstein et al., 2024), PQShield power-side-channel disclosures (PQShield, 2024), and SPA-CCA (Guo et al., 2024). They may attempt timing or power side-channels against the X25519 half of suite `0x06`. TesseraSeal mitigates software-timing leakage by pinning verified, constant-time third-party ML-KEM-1024 and X25519 implementations per language (sections 5 and 6; the only crypto primitive the monorepo *vendors* is Keccak/SHAKE-256, under `common/`) and documents an explicit "constant-time ≠ constant-power" disclaimer (section 11). Physical-access attacks (power, EM, frequency-scaling (Wang et al., 2022), prefetcher (Chen et al., 2024)) remain out of v1.x scope.

The attacker may attempt a *suite-confusion* attack: present an envelope with a suite byte that does not match the recipient's key, or re-tag a hybrid envelope as non-hybrid (or vice versa). TesseraSeal rejects this structurally at the API boundary (section 10): `encrypt` rejects a recipient key whose suite does not match the requested suite; `decrypt` rejects an envelope whose header suite does not match the private key's suite. Both are `BadKey`-class structural errors, distinguishable safely because they do not depend on secret-correlated bytes (enforced in all eight reference implementations as of 2026-06-10).

Residual risk and the v1.2 hybrid escape valve

TesseraSeal's **core promise** (suites `0x01`/`0x02`) is post-quantum-only by deliberate scope. If ML-KEM-1024 is later structurally broken (a sub-exponential Module-LWE attack), every `0x01`/`0x02` envelope loses confidentiality with no classical fallback. This 30–50 year risk is accepted in exchange for an unambiguous PQ-only promise; the mitigation path is forward-only via re-encryption under a future suite (section 15).

For callers who cannot accept that residual risk, v1.2 introduced the *opt-in* hybrid suite `0x06` (section 6). Suite `0x06` is IND-CCA2-secure as long as *either* ML-KEM-1024 *or* X25519 is IND-CCA2-secure; an attacker must break both to recover the session key. This is a separate suite allocation, not a modification of the core promise — the wire is unambiguous about which guarantee a given envelope makes.

3 Wire Format (Normative)

Big-endian throughout. **Header: fixed 16 bytes. Body: variable.**

```
Offset  Size  Field                                                AD?
------  ----  ---------------------------------------------------  ----
0       6     TS          (timestamp_ms; provenance, not auth)     yes
6       1     VAR         (variant byte = 0x04 for v1)             yes
7       1     SUITE       (algorithm suite id; see sec04)          yes
8       2     CTR         (counter, monotonic per sender)          yes
10      2     KEM_CT_LEN  (big-endian unsigned, <= 4096)            yes
12      4     AD_LEN      (length of caller-supplied AD)           yes
16      N     KEM_CT      (KEM ciphertext, N = KEM_CT_LEN)          yes
16+N    A     AD          (caller AD, A = AD_LEN)                   yes (carried)
16+N+A  12    AEAD_NONCE  (derived from K_session; see sec07)       no  (derived)
...     M     AEAD_CT     (AEAD_E(K_aead, NONCE, PT))              ---
```

```
...      16    AEAD_TAG    (128-bit tag over header || KEM_CT || AD)---
...      32    COMMIT      (key-commitment tag; HMAC-SHA3-256)      ---
```

The first 9 bytes (`TS || VAR || CTR`) are layout-compatible with TesseraID and TesseraHash: a single `common/c/tessera_header.h` pins those offsets across all three projects. Variant byte `0x04` is the first allocation from the previously-reserved `0x04–0xFE` range; TesseraID and TesseraHash decoders MUST reject `0x04`, and TesseraSeal decoders MUST reject every byte except `0x04` in v1.

Length limits. For an allocated suite, `KEM_CT_LEN` MUST equal that suite's KEM-ciphertext size: 1568 bytes for the pure ML-KEM-1024 suites `0x01`/`0x02`, 1600 bytes for the hybrid suite `0x06`. Decoders MUST reject any other value — including the 4096-byte field maximum — with an invalid-length error *before* attempting decapsulation; accepting a shorter declared length would let a fixed-size ML-KEM read run past the envelope buffer (the heap over-read fixed in TSEAL-00049). The 4096 field cap remains the structural upper bound that guards against oversized-allocation inputs. `AD_LEN` maximum 65 536 bytes. Plaintext maximum 2^{32} bytes (4 GiB); v1 is not a streaming format.

Wire form for envelopes (binary). v1 ships binary-only on the wire. Text encodings (Base64URL, Crockford Base32) are deferred to v1.1 because envelopes are variable-length and the encoder logic differs from TesseraID's and TesseraHash's fixed-length encoders. Callers that need text representations should wrap with standard Base64URL at the transport layer.

4 Algorithm Suites

The `SUITE` byte at offset 7 selects a complete algorithm bundle. v1.x allocates three:

Suite	KEM(s)	AEAD	KDF/MAC	Status	Notes
`0x01`	ML-KEM-1024	AES-256-GCM + UtC commit (32 B)	HKDF-SHA3-256	**mandatory**	v1 default, pure PQ
`0x02`	ML-KEM-1024	ChaCha20-Poly1305 + UtC commit (32 B)	HKDF-SHA3-256	optional	software-only, pure PQ
`0x06`	X25519 + ML-KEM-1024 (section 6)	AES-256-GCM + UtC commit (32 B)	HKDF-SHA3-256	optional	hybrid, v1.2 (PQCK-00021)
`0x07`	ML-KEM-1024+HQC or HQC-only	TBD	TBD	**reserved**	HQC contingency; reject in v1.x
`0x00`, `0x03`–`0x05`, `0x08`–`0xFF`	— reject —	—	—	—	reserved-future

HQC contingency (suite `0x07` — RESERVED, not allocated). Suite byte `0x07` is *reserved* for the ML-KEM-failure contingency: an ML-KEM-1024+HQC hybrid or an HQC-only successor suite built on NIST's selected backup KEM, HQC (National Institute of Standards and Technology, 2025). The byte will be *allocated* upon NIST HQC FIPS finalization (the draft standard has slipped; final expected ∼2027). The *activation triggers* for actually shipping it are (a) a loss of confidence in the Module-LWE problem underlying ML-KEM, or (b) HQC standardization together with verified, constant-time HQC implementations across the pinned per-language sources. Until allocation, `0x07` remains reserved-future on the wire and every v1.x decoder MUST reject it like any other unallocated byte — the reservation is a registry entry, not wire behavior. When allocated, `0x07` follows the standard new-suite discipline: whitepaper amendment, Python reference first, lockstep ports, frozen KAT vectors. This converts a structural ML-KEM break from an unplanned emergency into a pre-reserved migration path, complementing the X25519 hybrid escape valve of suite `0x06`: code-based HQC does not share ML-KEM's lattice assumptions, so it remains a viable post-quantum fallback even in the Module-LWE-loss scenario where `0x06` degrades to classical-only protection.

The wire is unambiguous about which guarantee a given envelope makes: suites `0x01`/`0x02` commit

to "post-quantum confidentiality, no classical fallback"; suite `0x06` commits to "post-quantum *or* classical confidentiality, the attacker must break both." Suite `0x06` is a *separate suite-byte allocation* (section 6); it is never an in-place modification of `0x01`/`0x02`. Callers select by passing the suite byte explicitly to `encrypt`; `decrypt` dispatches by the suite byte in the envelope header.

Why ML-KEM-1024 only, not ML-KEM-768? The core promise (`0x01`/`0x02`) has no hybrid safety net (see section 2). We buy the largest NIST-defined PQ margin (level 5, ∼AES-256-equivalent classical-search) at the cost of a 1568 B KEM ciphertext (vs 1088 B for ML-KEM-768). For envelope use this ∼500 B difference is irrelevant; the security margin is not. Suite `0x06` keeps ML-KEM-1024 (not 768) for the same reason: even with X25519 as a backup, the PQ half of the hybrid should not be downgraded.

Why AES-256-GCM + UtC, not AEGIS or GCM-SIV? AEGIS-128L is 2–3× faster on modern x86 and ARM (rfc, 2025) but its RFC finalized only September 2025; library support is immature. AES-256-GCM has decade-scale review, AES-NI/VAES acceleration to ∼10 GB/s (Intel Corporation, 2025; Bernstein and Lange, 2025), and a settled commitment construction in UtC (Bellare and Hoang, 2022). GCM-SIV duplicates the misuse-resistance property that the deterministic-nonce-from-KDF already provides structurally (section 8). AEGIS-128L was evaluated for v1.2 (a hypothetical suite `0x03`) in PQCK-00020 and *deferred*: as of the v1.2 evaluation (2026-05-21), AEGIS implementations were not yet polyglot-ready across Python, Go, Java, and OpenSSL. The byte remains reserved-future; revisit when verified implementations land across libcrux, OpenSSL, and the Go stdlib.

Why HKDF-SHA3-256? SHAKE/Keccak is already vendored under `common/c/keccak.{h,c}` for TesseraHash; HKDF-SHA3 reuses that primitive and aligns with FIPS 203's internal use of Keccak. SHA-256 with SHA-NI is faster but creates two hash families to audit and certify.

No agility within a suite. A suite byte *is* a version: changing any algorithm requires a new suite byte. There is no negotiation. This matches TesseraID's and TesseraHash's immutable-variant-byte discipline and avoids the downgrade and cross-protocol attack surface of TLS-style negotiation (cf. Terrapin (Bäumer et al., 2023)).

5 ML-KEM-1024 Binding

TesseraSeal invokes ML-KEM-1024 unchanged from FIPS 203 (fip, 2024): `KeyGen`, `Encaps(pk)` producing (K, c), and `Decaps(sk, c)` producing K (or an implicit-reject pseudorandom value on malformed c, per the Fujisaki–Okamoto wrapper). We do not modify the FO transform, do not add a combiner, and do not wrap ML-KEM in any further construction. The shared secret K from `Decaps` is fed directly to `HKDF-Extract` as IKM (section 8).

5.1 Tight QROM proof

ML-KEM-1024's IND-CCA2 security has a tight QROM proof under Module-LWE (Hofheinz et al., 2017; Don et al., 2022). TesseraSeal inherits this directly. We make no assumption beyond what FIPS 203 already requires.

5.2 Implementation sources (v1.x, as shipped)

Language	ML-KEM-1024 source (pinned)	Constant-time claim
Python	`kyber-py >= 1.0`	vendor-documented; NCC Group audit Q2 2024
C	**OpenSSL 3.5+** `EVP_KEM` ML-KEM-1024	OpenSSL constant-time path; FIPS 140-3 candidate
Go	**Go 1.26 stdlib** `crypto/mlkem`	stdlib constant-time
Rust	`ml-kem = "0.3"` (RustCrypto, `MlKem1024`)	const-time documented; `#![forbid(unsafe_code)]`
C++	wraps the C library	inherits OpenSSL
TypeScript	`@noble/post-quantum`	const-time documented; pure JS
Java	**SunJCE** JEP 452 (*not* BouncyCastle)	JDK stdlib constant-time
CPython ext	wraps the C reference (OpenSSL 3.5+ `EVP_KEM`)	inherits OpenSSL

Decisions worth flagging:

- *C uses OpenSSL 3.5+ `EVP_KEM`, not `mlkem-native`.* The earlier draft of this whitepaper named `mlkem-native` (Cryspen and PQ Code Package, 2025); the PQCK-00005 release note records the decision to switch. OpenSSL has a smaller audit surface for our deployment (we already link `libcrypto` for the AEADs), a FIPS 140-3 validation path, and ubiquitous packaging. `mlkem-native` is *not* vendored, linked, or retained by this project in any capacity; an embedded target that cannot link OpenSSL would have to bring its own ML-KEM source, which requires the security review mandated by the supply-chain rules in `tessera-seal/CLAUDE.md` and is outside v1.x scope.
- *Java uses SunJCE (JEP 452), not BouncyCastle.* The JEP 452 `KEM` API landed in JDK 24 and provides ML-KEM-1024 as a first-class `KeyPairGenerator` algorithm. BouncyCastle 1.79+ remains a viable alternative but is not the v1.1 reference; this avoids the CVE-2025-1432 surface (section 11) in the default vendoring story.
- *Go uses stdlib `crypto/mlkem` (Go 1.26+), not `filippo.io/mlkem`.* The earlier external library was the v1 alpha intent; by the time v1.1 shipped, ML-KEM had landed in the Go stdlib and the dependency was eliminated.

The v1 alpha language set (Python, C, Go, Rust) is the minimum that exercises the spec across a readable reference (Python), a canonical ABI (C), a stdlib-heavy ecosystem (Go), and the strongest constant-time toolchain (Rust). C++, TypeScript, Java, and CPython ext joined in v1.1; see Appendix F for the release timeline.

5.3 The "Kyber" naming ban

ML-KEM (FIPS 203, August 2024) is wire-incompatible with the Kyber-r3 specification it descends from. TesseraSeal's public APIs MUST NOT expose the token "Kyber" even where the upstream library uses it. Rename at the binding layer (e.g., `kyber-py.ML_KEM_1024` → `tessera_seal.MLKem1024`). The same discipline applies to the hybrid suite's classical half: only "X25519" appears in user-facing identifiers, never "Curve25519" or "ECDH."

6 Hybrid Suite `0x06` (Optional)

Suite `0x06` (v1.2, PQCK-00021) is the opt-in hybrid alternative to the pure-PQ suites. It is a *separate suite-byte allocation*, never an in-place modification of `0x01` or `0x02`: callers who want the unambiguous PQ-only brand promise of section 2 keep using `0x01`/`0x02`; callers who want a classical safety net against a Module-LWE structural break opt in to `0x06`. Both promises remain available, and the wire is unambiguous about which one a given envelope makes.

6.1 Construction

Suite `0x06` runs ML-KEM-1024 and X25519 (Langley et al., 2016) as independent KEMs, then combines them with a SHAKE-256 (fip, 2015) combiner inspired by X-Wing (Connolly et al., 2025):

$$\begin{aligned}(K_{\text{mlkem}}, c_{\text{mlkem}}) &\leftarrow \text{ML-KEM-1024.Encaps}(pk_{\text{mlkem}})\\ (sk_{\text{eph}}, pk_{\text{eph}}) &\leftarrow \text{X25519.GenerateKeypair}()\\ K_X &\leftarrow \text{X25519.ECDH}(sk_{\text{eph}}, pk_{\text{x}})\end{aligned}$$

The session shared secret is:

$$K_{\text{session}} = \text{SHAKE-256}(\text{“pqcrypt v1 hybrid combiner”} \,\|\, K_{\text{mlkem}} \,\|\, K_X \,\|\, c_{\text{mlkem}} \,\|\, pk_{\text{eph}},\ 32 \text{ bytes})$$

(The literal label `"pqcrypt v1 hybrid combiner"` is a frozen v1 wire tag hashed into the envelope; it is not a rebrandable product name and is preserved verbatim across the rename to TesseraSeal.) K_{session} replaces `ss_mlkem` as the IKM for the HKDF schedule of section 8; the rest of the wire format and AEAD path is unchanged. `KEM_CT` on the wire carries $c_{\text{mlkem}} \,\|\, pk_{\text{eph}}$, total 1600 bytes (1568 + 32).

6.2 Sizes

Object	Size (bytes)	Composition
Hybrid public key	1 600	ML-KEM-1024 `pk` (1568) ‖ X25519 `pk` (32)
Hybrid private key	3 200	ML-KEM-1024 `sk` (3168) ‖ X25519 scalar (32)
Hybrid `KEM_CT`	1 600	ML-KEM-1024 `ct` (1568) ‖ X25519 ephemeral `pk` (32)
Hybrid seed (KAT reproducibility)	96	FIPS 203 ($d \,\|\, z$) (64) ‖ X25519 scalar (32)

6.3 Security argument

The combiner is an IND-CCA2 KEM iff *either* component KEM is IND-CCA2-secure under the assumption that SHAKE-256 is a random oracle for the combiner's input. This is the conservative “OR-style” hybrid: an attacker must break *both* ML-KEM-1024 *and* X25519 to recover K_{session}. The conservative direction matches X-Wing (Connolly et al., 2025) and is the consensus design for transport hybrids (TLS 1.3 `X25519MLKEM768`, OpenSSH `mlkem768x25519-sha256`, MLS RFC 9420 `X25519MLKEM768`). Including *both* ciphertext halves (c_{mlkem} and pk_{eph}) in the combiner input prevents an attacker who controls one half from forcing a known K_{session} even with full control of the other half.

6.4 Cost

One additional X25519 keygen plus one X25519 ECDH per envelope on encrypt; one X25519 ECDH per envelope on decrypt. On Apple M4 Max with the Go stdlib `crypto/ecdh`, the per-envelope amortized cost is approximately 50 µs on top of the ML-KEM-1024 baseline (section 14). `KEM_CT` grows by 32 bytes per envelope.

6.5 Constant-time and supply-chain posture

The X25519 implementation must be constant-time on the scalar multiplication. Verified per language: `x25519-dalek` v2 (Rust, `#![forbid(unsafe_code)]`), Go stdlib `crypto/ecdh` (constant-time on aarch64 and x86_64), Python `cryptography` (libsodium-backed), and OpenSSL 3.5+ `EVP_PKEY_X25519` (C — the same pinned OpenSSL that supplies ML-KEM, no new dependency). Hybrid-capable language ports: the harness-enforced 4-way byte-identical KAT contract covers Python (PQCK-00021, v1.2), Rust (PQCK-00028, v1.3), Go (PQCK-00029, v1.3), and C (TSEAL-00030, 2026-06-13). Library-level hybrid additionally ships in C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034), each verified by an in-suite test that decodes the frozen `hybrid_*.json` vectors byte-identically — they implement suite `0x06` even though they have no TesseraSeal conformance runner yet. Java (TSEAL-00033, backlog) is the sole remaining port that still rejects suite `0x06` as `UNKNOWN_SUITE`. See section 13 for the harness-level skip discipline (which is a property of the runner, not of library support).

When to opt in to suite `0x06`

Use suite `0x06` when the deployment's threat model includes "Module-LWE may turn out to be structurally weaker than believed" and the operator wants belt-and-suspenders coverage. Use suites `0x01`/`0x02` when the deployment wants an unambiguous post-quantum confidentiality promise (FIPS 203 NIST level 5 alone) and treats the residual Module-LWE risk as acceptable (section 2). Both promises remain on the wire; the suite byte names which one the envelope makes.

7 AEAD and Commitment

7.1 Why a separate commitment tag

AES-256-GCM and ChaCha20-Poly1305 are confidentiality-and-integrity AEADs but *not* key-committing (Grubbs et al., 2017): an attacker with two different keys can produce a single ciphertext that decrypts correctly under both. Weaponized variants (Albertini et al., 2020; Len et al., 2021) matter most in multi-recipient settings where the same envelope reaches recipients holding different keys — e.g., the exact class of bug `age` still has open in 2026. TesseraSeal closes this with an explicit 32-byte commitment tag.

7.2 The UtC construction (Bellare & Hoang 2022)

Given the HKDF pseudorandom key `prk` from section 8, derive two subkeys (the literal HKDF-Expand info labels below are frozen v1 wire tags; see section 8):

$$K_{\text{aead}} = \text{HKDF-Expand}(\texttt{prk}, \text{“pqcrypt v1 aead k”}, 32)$$
$$K_{\text{commit}} = \text{HKDF-Expand}(\texttt{prk}, \text{“pqcrypt v1 commit”}, 32)$$

Encrypt under K_{aead} as usual, producing `AEAD_CT` and `AEAD_TAG`. Then compute:

$$\text{COMMIT} = \text{HMAC-SHA3-256}(K_{\text{commit}},\ \text{header} \,\|\, \text{KEM_CT} \,\|\, \text{AD} \,\|\, \text{AEAD_CT})$$

The 32-byte `COMMIT` appears as the final field on the wire (section 3).

7.3 Verification

Decryption verifies both `AEAD_TAG` (covering header, `KEM_CT`, AD, `AEAD_CT`) and `COMMIT` (covering header, `KEM_CT`, AD, `AEAD_CT`) in constant time before returning the plaintext. On decrypt, the AEAD AAD is reconstructed as header $\|$ KEM_CT $\|$ AD where `AD` is the *on-wire* associated data carried in the envelope; the caller's `expected_ad` is compared against the on-wire AD with a constant-time equality check, and that equality check is the substitution guard. This AAD construction is uniform across all eight reference implementations (TSEAL-00042). On authentication-class failures the implementations run the synthetic-key timing pad of section 10; structural errors (truncated input, unknown variant or suite, BadKey suite mismatch) MAY return early, as defined there.

7.4 Why UtC over CTX, GCM-SST, or compactly-committing schemes

CTX (Chan and Rogaway, 2022) is gaining adoption (libsodium 1.0.21, MLS) and shares UtC's tag-then-commit shape. UtC was chosen for v1 because (a) the original Bellare–Hoang proof is the most cited reference in subsequent literature, (b) its construction is simplest to implement in eight languages without crypto-library help, and (c) the 32-byte overhead matches our wire-format budget comfortably. Succinctly-committing AE designs that shorten the committing tag (Bellare and Hoang, 2024) — and the related GCM-SST proposal — remain without polyglot library support as of 2026. Compactly-committing AEADs (Grubbs et al., 2017) are more space-efficient but introduce a non-standard cipher dependency.

7.5 Commitment vs content binding

The UtC commitment binds the AEAD key to the ciphertext — it answers the question “can the same ciphertext open to different plaintexts under different keys?” (no, by the commitment). It does *not* answer “does this ciphertext correspond to a specific known plaintext content?”. That question is answered by binding a TesseraHash (the SHAKE-256-based content-hash sibling format under the tessera monorepo; variant byte `0x03`) as the AEAD AD: a TesseraSeal envelope whose AD is the TesseraHash of the plaintext is cryptographically bound to that specific content. The two bindings are independent and stack cleanly: UtC closes multi-recipient confusion; TesseraHash-as-AD closes content substitution. See section 9 for the AD-binding patterns including TesseraID-as-AD and TesseraHash-as-AD.

8 Key Derivation and Deterministic Nonce

8.1 Key schedule

Let `ss_session` be the 32-byte session shared secret. For suites `0x01`/`0x02` it is `ss_mlkem` (the value returned by ML-KEM-1024 `Decaps`, or `Encaps` on the sender side); for suite `0x06` it is the SHAKE-256 combiner output defined in section 6. The downstream schedule is identical:

$$\begin{aligned}
\text{salt} &= \text{SHAKE256}(\text{“pqcrypt v1 header”} \,\|\, \text{header_bytes}[0{:}16],\ 32) \\
\text{prk} &= \text{HKDF-Extract}(\text{salt},\ \texttt{ss_session}) \\
K_{\text{aead}} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 aead k”},\ 32) \\
\text{NONCE} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 aead n”},\ 12) \\
K_{\text{commit}} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 commit”},\ 32)
\end{aligned}$$

The PRF is HMAC-SHA3-256 (fip, 2015). The same label strings are used for every suite — the suite byte enters the schedule via the salt (which binds the full header, including the suite byte) rather than via separate labels, so an attacker who tries to re-tag a `0x01` envelope as `0x06` (or vice versa) gets a different `salt`, a different `prk`, and AEAD verification failure.

Frozen v1 wire tags

The literal domain-separation labels `"pqcrypt v1 header"`, `"pqcrypt v1 aead k"`, `"pqcrypt v1 aead n"`, `"pqcrypt v1 commit"` (and, for the hybrid suite, `"pqcrypt v1 hybrid combiner"` of section 6, plus the `"pqcrypt v1 timing pad"` of section 10) are **frozen v1 wire constants**: they are byte strings hashed *into* every envelope. They are deliberately *not* rebrandable product names and are preserved verbatim — including the legacy token `pqcrypt` — across the rename to TesseraSeal. Changing any of them would break every existing envelope and conformance vector. They are the only place the token `pqcrypt` legitimately remains.

8.2 Salt binding

The salt mixes a 16-byte project-and-version constant with the full envelope header (timestamp, variant, suite, counter, KEM CT length, AD length). Any header bit-flip therefore invalidates K_{aead} before the AEAD tag is even checked, providing a second line of defense against header tampering. The constant prevents cross-protocol reuse of the KDF.

8.3 Why deterministic nonce

Each ML-KEM-1024 encapsulation produces a fresh ephemeral shared secret, so K_{aead} is unique per envelope. For hybrid suite `0x06` (section 6), the X25519 ephemeral keypair is also freshly generated per envelope and the SHAKE-256 combiner produces a fresh K_{session} regardless of which component KEM is broken. Deterministic derivation of `NONCE` from `prk` therefore cannot cause nonce reuse under the same key on any suite. This:

- Removes the encrypt-side CSPRNG dependency beyond ML-KEM's internal `Encaps` randomness.
- Matches PASETO v4.local's philosophy (Arciszewski, 2021): deterministic derivation under a

per-message fresh key is structurally safer than per-message random nonce generation.

- Eliminates a class of misuse: an implementer who accidentally reuses a CSPRNG seed cannot create a nonce-collision oracle, because the seed cannot survive ML-KEM `Encaps` (fip, 2024).

8.4 Multi-user / multi-key bounds

The single-key birthday bound on AES-256-GCM is 2^{64} blocks; TesseraSeal's plaintext cap of 2^{32} bytes per envelope keeps each invocation well below the bound. The multi-user attack analysis of Bose, Hoang, and Tessaro (Bose et al., 2018) concerns adversaries who attack many keys simultaneously; TesseraSeal's per-envelope fresh K_{aead} removes the long-key-lifetime axis of that attack class.

9 The TesseraID-as-AD Canonical Pattern

TesseraSeal's existence as a sibling of TesseraID and TesseraHash in the same monorepo is not aesthetic — it is structural. The canonical usage pattern is to bind a TesseraID as the AEAD's associated data:

```
record_id = tessera_id.new_hlc()                          # 41 bytes
envelope  = tessera_seal.encrypt(plaintext, recipient_pub,
                                 associated_data=record_id)
# ... later, after delivery ...
plaintext = tessera_seal.decrypt(envelope, recipient_priv,
                                 expected_ad=record_id)   # MUST match
```

What this defends. An attacker who reroutes the envelope under a different TesseraID cannot decrypt it: AEAD tag verification fails (AD mismatch) and the decryption path returns the generic `DecryptionError` (section 10). The envelope is cryptographically bound to the identifier it was issued for.

Cost. Free. Every AEAD's AD path is computed at no marginal cost beyond the bytes being mixed into the MAC. TesseraSeal's bench measurement `encrypt_with_tessera_id_ad` confirms this experimentally (section 15).

Cross-project AD patterns. Other useful AD choices:

- A TesseraHash of the plaintext, for envelopes that explicitly bind to content (defense against substituting one record's envelope onto another record).
- A concatenation of a TesseraID and a recipient identifier, for envelopes addressed to a specific (record, recipient) pair.
- A protocol-level domain separator (e.g., `"tessera-seal-app-v1-records"`) plus the TesseraID.

Conformance. The KAT file `tessera-seal/conformance/vectors/ad_binding.json` exercises positive cases (envelope decrypts with the matching AD) and negative cases (any single-bit AD change causes a generic `DECRYPT_FAILED`). The KAT asserts the error class only; decrypt-path timing posture is described in section 10 and section 11.

10 Failure Modes

10.1 One error class for every authentication-class failure

TesseraSeal's `decrypt` API returns exactly one error category for every failure where the attacker could be probing for an oracle: `DecryptionError` (Python, TypeScript, Java) / `TESSERA_SEAL_E_DECRYPT_FAILED` (C ABI) / `Error::Decrypt` (Rust). The categories that map to this single error:

- Bad AEAD tag.
- Bad UtC commitment tag.
- Bad KEM ciphertext (ML-KEM implicit-reject path).
- AD mismatch.
- Unknown variant inside a decrypted internal blob (reserved for future suite-version embeds).

The categories that map to distinct *structural* errors and may be distinguished safely:

- Truncated envelope or impossible length field (`INVALID_LEN`).
- Unknown variant byte at offset 6 (`UNKNOWN_VARIANT`).
- Unknown suite byte at offset 7 (`UNKNOWN_SUITE`). A port that does not implement suite `0x06` rejects it here: at `peek_header` *and* at `decrypt`, and ordered *before* the `BAD_KEY` suite/key-consistency check below. As of 2026-06-13 the sole such port is Java (TSEAL-00033, backlog); its rejection ordering is pinned by a per-language unit test (TSEAL-00044). The ports that historically rejected `0x06` — C (TSEAL-00030), C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034) — now implement suite `0x06` at the library level and accept it (section 13).
- Suite/key mismatch (`BAD_KEY`). `encrypt` rejects when the recipient public key's suite does not match the requested suite; `decrypt` rejects when the private key's suite does not match the envelope's header suite. This is structural — it does not touch secret-correlated bytes — and distinguishable safely. (Enforced in all eight reference implementations as of 2026-06-10.)
- Output buffer too small (C ABI only: `BUF_TOO_SMALL`).

10.2 Constant-time discipline

On every *authentication-class* failure path (bad AEAD tag, bad commit tag, KEM implicit-reject, AD mismatch), the decrypt routine MUST run the synthetic-key schedule — the HKDF derivations of section 8 performed against a synthetic key and then discarded — as a timing pad, so the failure path performs the same key-derivation work as the success path. The synthetic schedule derives from `HKDF_Expand(prk_zero, "pqcrypt v1 timing pad", 32)` where `prk_zero` is a 32-byte all-zero buffer. The literal label `"pqcrypt v1 timing pad"` is a frozen v1 wire tag (section 8) and is preserved verbatim across the rename to TesseraSeal.

The Python reference additionally executes a synthetic AEAD verify on the KEM-decaps-failure path. *Structural* errors — invalid length, unknown variant, unknown suite, `BAD_KEY` suite mismatch — MAY return early without running the synthetic schedule: they are caller errors that are distinguishable by

design (see the structural-error list above), and their detection touches no per-key secret-correlated bytes, so an early return discloses nothing beyond the error class itself. The C reference's failure-path comment was aligned to this implemented behavior in TSEAL-00043. (An earlier revision of this section required a synthetic AEAD verify *even on structural failure paths*; no port implemented that stronger wording, and this revision replaces it with the contract the eight implementations actually share.)

10.3 Timing measurement (automated script; not yet a blocking gate)

An automated timing-histogram regression ships at `tools/scripts/timing-histogram.py` (TSEAL-00048): it measures N good-vs-bad decrypt pairs against the Python reference (default $N = 20\,000$; "bad" flips one bit in the on-wire commitment tag, so both sides execute the full KEM-decaps + KDF + AEAD path), reports median and p99 deltas, writes a JSON artifact for cross-run trend tracking, and asserts a soft threshold on the median per-pair delta (warn above 50 ns, hard-fail above 500 ns, with a `-strict` flag). It runs as a dispatch-only job in the manual `tessera-seal.yml` workflow; it is **not yet a blocking PR gate**. On contended hosts the single-run noise floor of this statistic is on the order of $\pm 1\,\mu$s (initial 2026-06-10 runs on a heavily loaded Apple M4 Max measured medians from -0.65 to $+1.4\,\mu$s with sign flips across runs — noise, not signal); read the artifact as a trend point. No timing dataset and no timing assertion exist under `tessera-seal/conformance/vectors/` — the regression is a script, not a conformance vector.

The most recent *manual* measurement (Python reference, 2026-05-19, Apple M4 Max, quiet host) compared 100 000 decrypt pairs (50 000 good envelopes, 50 000 single-bit-flipped) and observed a median good-vs-bad latency difference below 50 ns; the procedure is recorded in `tessera-seal/docs/security-audit-v1.2` (open item #1 — the script above addresses the automation half of that item; promotion to a blocking CI gate remains open). Per-language measurement environments are recorded under `tessera-seal/benchmarks/`.

10.4 Implicit-reject semantics

The FO transform in ML-KEM-1024 (fip, 2024; Hofheinz et al., 2017) returns a pseudorandom 32-byte value on malformed ciphertext rather than raising an error. TesseraSeal preserves this: a bad `KEM_CT` produces an implicit-reject `ss_mlkem`, which is then expanded into a useless K_{aead}, which then fails AEAD tag verification, which finally produces the generic `DecryptionError`. The attacker cannot distinguish "bad KEM CT" from "bad AEAD tag" from "bad commit tag" from "AD mismatch."

11 Side Channels

11.1 Known attacks 2024–2026

- **KyberSlash 1 & 2** (Bernstein et al., 2024), Dec 2023 / Jan 2024: division-by-secret timing leak in ML-KEM `Decaps` and `poly_compress`. Patched in pq-crystals reference and liboqs 0.10 in January 2024. TesseraSeal pins only post-patch implementations.
- **PQShield power-analysis disclosure** (PQShield, 2024), Mar 2024: SPA on Cortex-M4 ML-KEM `Decaps` recovering the secret in fewer than 1000 traces. Mitigations live in masked

implementations (e.g. `mlkem-native`'s masking branch); no library pinned by v1 enables masking.

- **SPA-CCA on ML-KEM** (Guo et al., 2024), ePrint 2024/2051: single-trace CCA on unmasked Cortex-M4. Masked implementations unaffected.
- **CVE-2024-39427** (liboqs $< 0.10.1$, Jul 2024): ML-KEM `Decaps` timing on certain platforms. Fixed pre-v1.
- **CVE-2025-1432** (Bouncy Castle 1.78, Feb 2025): ML-KEM rejection-sampling bias on malformed public keys. Fixed in BouncyCastle 1.79+. The v1.1 Java port avoids the surface entirely by using SunJCE JEP 452 (JDK 24+) as the ML-KEM source rather than BouncyCastle (section 5); a BouncyCastle-backed alternative remains usable but must pin $\geq$ 1.79.

11.2 What v1 mitigates

TesseraSeal v1.x mitigates software-timing side channels by:

- Pinning verified, constant-time third-party ML-KEM-1024 implementations per language (section 5); none of them is vendored into this repository (only the Keccak/SHAKE-256 primitive is, under `common/`). For hybrid suite `0x06` (section 6), the X25519 source is similarly pinned per language to a constant-time implementation: `x25519-dalek` v2 in Rust (`#![forbid(unsafe_code)]`), Go stdlib `crypto/ecdh` (constant-time on aarch64 and x86_64), Python `cryptography` (libsodium-backed). Small-subgroup / low-order point inputs are rejected structurally by the X25519 library and surface as `DecryptionError` via the constant-time decrypt wrapper.
- Constant-time AEAD tag and commitment tag verification (sections 7 and 10).
- A single `DecryptionError` class covering every authentication-related failure.
- Timing-histogram measurement of the decrypt path (section 10): an automated script ships at `tools/scripts/timing-histogram.py` (TSEAL-00048) and runs in the manual `tessera-seal.yml` dispatch; it is not yet a blocking PR gate. No timing vectors ship in conformance today.

11.3 What v1 does *not* mitigate

Constant-time is not constant-power

TesseraSeal v1 does not mitigate power-analysis, electromagnetic-emanation, frequency-scaling (Wang et al., 2022; Chen et al., 2024), or prefetcher (Chen et al., 2024) side channels. Deployment in environments where an attacker can measure CPU power, EM, frequency, or memory-prefetcher behavior — smart cards, embedded devices, shared multi-tenant hosts without DOIT-class mitigations (Intel Corporation, 2022) — requires additional countermeasures outside the library's scope.

The companion document `tessera-seal/docs/side-channel-disclaimer.md` restates these limits in operator-facing language and links to recommended hardware-isolation patterns for high-assurance

deployments.

11.4 Forward path

A future suite byte (drawn from the remaining reserved-future range; suite `0x06` is now taken for hybrid, section 6) could pin a masked ML-KEM-1024 implementation as mandatory for high-assurance deployments where power-analysis is in scope. The variant-byte and suite-byte allocation discipline supports this without breaking v1.x envelopes (section 15).

12 Plugin Protocol

12.1 Why a plugin protocol in v1

TesseraSeal's core spec does *not* define a key-on-disk format. Specifying one in v1 would require choosing a passphrase KDF (Argon2id parameters), a wrapping cipher, a versioning byte, and a textual armor format — each a non-trivial decision that ages on a different clock than the wire format itself. The lesson from `age`'s ecosystem is that this work belongs to plugins (`age-plugin-yubikey`, `age-plugin-tpm`, `age-plugin-se`, `age-plugin-fido2`). TesseraSeal ships the plugin protocol so plugin authors can build immediately, plus a reference plugin (`tessera-seal-plugin-file`, section 12.3) so the v1 spec is self-contained.

12.2 Wire format

The plugin protocol uses stdin/stdout binary frames modeled on the C2SP `age-plugin` specification (C2SP, 2023). The TesseraSeal library invokes `tessera-seal-plugin-<name>` via `exec`, passes envelope-level data through framed messages, and receives wrapped/unwrapped key material in return.

Frames are length-prefixed: 4-byte big-endian length, then the frame body. Frame types are identified by a leading single-byte tag. v1 frame types:

- `RECIPIENT_STANZA` (`0x01`): declares the plugin's recipient identifier in URI form `tseal1<plugin-name>1<base`
- `WRAP_KEY` (`0x02`): library → plugin, requesting wrap of a per-envelope file-key.
- `UNWRAP_KEY` (`0x03`): library → plugin, requesting unwrap of a previously-wrapped file-key.
- `OK` (`0x10`): plugin → library, success with body payload.
- `ERROR` (`0x11`): plugin → library, failure with optional explanatory string.

The full state machine, including handshake and pipelining rules, is defined in `tessera-seal/plugin/tessera-seal`

12.3 Reference plugin: `tessera-seal-plugin-file`

`tessera-seal-plugin-file` stores a passphrase-wrapped ML-KEM-1024 private key on disk. The current on-disk format is `tseal-key-v1.1` (PQCK-00018, v1.1); the older `tseal-key-v1` format remains readable for backwards compatibility. Construction:

1. Read the passphrase from the terminal (no echo) or from `$TESSERA_SEAL_PASSPHRASE`.

2. Derive a 32-byte symmetric key: `K_wrap = Argon2id(passphrase, salt, m=64MiB, t=3, p=1, 32 bytes)` (rfc, 2021).
3. Wrap the ML-KEM-1024 private key (∼3168 B; or 3200 B for a hybrid keypair, section 6) under AES-256-GCM: `wrapped = AES-256-GCM-Encrypt(K_wrap, nonce, sk)` where `nonce` is 12 fresh CSPRNG bytes.
4. Serialize: `tseal-key-v1.1\n<base64url(salt ‖ nonce ‖ public_key ‖ wrapped ‖ tag ‖ kdf_block)>\n`.

The KDF block (TSEAL-00040). The `tseal-key-v1.1` blob ends with an explicit, authenticated 13-byte KDF block: `kdf_id(1) || p0(4) || p1(4) || p2(4)`, all big-endian. `kdf_id 0x01` = Argon2id with `p0` = memory (KiB), `p1` = time cost, `p2` = parallelism; `kdf_id 0x02` = PBKDF2-HMAC-SHA-512 with `p0` = iterations and `p1` = `p2` = 0. The AES-256-GCM key-wrap AAD for files carrying the block is `header || kdf_block`, so on-disk tampering with the KDF id or parameters fails the wrap tag. Wrapping fails closed when Argon2id is unavailable: the plugin refuses to write a PBKDF2-wrapped keyfile unless the caller explicitly opts in (`-allow-pbkdf2`); no environment probe ever selects the KDF, and unwrap derives with exactly the KDF named in the file (an Argon2id-wrapped file hard-fails where Argon2id is unavailable, never silently retried under PBKDF2). Keyfiles predating the KDF block (`tseal-key-v1` and early `tseal-key-v1.1`, distinguished by header plus exact blob length) remain readable via a legacy derive path. This is plugin-protocol behavior, *not* part of the core wire format (section 3).

Why v1.1 embeds the public key. A v1.0 keyfile required passphrase unwrap before the plugin could answer a `pubkey` query, which the CLI binary (PQCK-00022, see below) exercises on every recipient lookup. v1.1 stores the unencrypted public key alongside the wrapped private key so plugins can answer `pubkey` without prompting for a passphrase. The public key was never secret; only the private-key wrap is.

Reference CLI. A reference CLI binary ships in v1.2 (PQCK-00022): `python -m tessera_seal.cli {keygen,pubkey,encrypt,decrypt,info}`. It invokes `tessera-seal-plugin-file` via subprocess for every key-custody operation; no private key material is ever read from `argv`.

The reference plugin is not part of the v1 normative wire format; only the plugin *protocol* is. Other plugins (`tessera-seal-plugin-yubikey`, hypothetical) may wrap keys however they want.

12.4 What the protocol does *not* cover

- Key custody (HSM, secure enclave, TPM): a job for plugin authors.
- Key rotation: a higher-level concern; TesseraSeal envelopes are self-contained and do not encode rotation lineage.
- Plugin trust: TesseraSeal invokes whichever plugin binary the caller's PATH resolves; the caller is responsible for trusting that binary.

13 Conformance

13.1 KAT vector files

`tessera-seal/conformance/vectors/`. The listing below matches the shipped directory contents as of 2026-06-13 (post the 2026-06-10 KAT expansion, TESS-00009, and the TSEAL-00049 `KEM_CT_LEN` pin):

- `decrypt.json` (9 records) — known envelopes → known plaintexts, exercising the decrypt path for suites `0x01` and `0x02`, including a 64 KiB-plaintext record (`pt_64kib_aesgcm`) and a maximum-`AD_LEN` (65 536-byte) record (`ad_max_65536_aesgcm`).
- `peek_header.json` (5 records) — header parsing without authentication, including counter-boundary records at `CTR` = `0xFFFE` and `0xFFFF`.
- `invalid.json` (32 records) — bad variant bytes (`0x00`–`0x03`, `0x05`, `0x10`, `0x7F`, `0xFE`, `0xFF`); bad suite bytes (`0x00`, `0x03`–`0x05`, `0x10`, `0x7F`, `0xFE`, `0xFF`); single-bit flips in `KEM_CT`, AD, nonce, `AEAD_CT`, `AEAD_TAG`, and `COMMIT`; the `bad_key_suite_mismatch` record (BadKey class, section 10); truncation with a structurally valid header (mid-`KEM_CT`, mid-`AEAD_CT`, one byte short of `COMMIT`); and two wrong-`KEM_CT_LEN`-with-valid-header records (one declaring the wrong length, one declaring a too-small length that would trigger the CWE-125 heap over-read fixed in TSEAL-00049).
- `ad_binding.json` (6 records) — TesseraID-as-AD positive and negative cases.
- `commitment.json` (3 records) — UtC commit tag KATs independent of the AEAD path.
- `min_timestamp.json` (4 records) — `MIN_TIMESTAMP_MS` helper parity (mirrors TesseraID's and TesseraHash's analogous files).
- `hybrid_decrypt.json` (4 records) — KAT round-trip for hybrid suite `0x06`.
- `hybrid_ad_binding.json` (4 records) — hybrid AD-binding positive and negative cases.
- `hybrid_peek_header.json` (1 record) — hybrid header parse.

Two vector files are *planned* (backlog) and are *not* part of the shipped vector set:

- `encrypt_roundtrip.json` — fixed-RNG-seed encrypt determinism. Each record would give a plaintext, a recipient public key, a sender RNG seed, and an expected envelope hex. The generator pathway exists (the seeded DRBG described under "Vector generation" below) but the file has not shipped.
- A KyberSlash-class timing-regression vector (working name `kyberslash_regression.json`) — known-bad ML-KEM-1024 inputs that exercised KyberSlash 1/2 (Bernstein et al., 2024), asserting they reject without timing variance.

13.2 Cross-language runner

Vectors are consumed by the same harness used by TesseraID and TesseraHash: `tools/conformance/run.sh`. Dedicated per-language TesseraSeal runners exist *only* for Python, C, Go, and Rust (`tools/conformance/python/`, `.../c/`, `.../go/`, `.../rust/`); the harness executes each against each vector file, and exit code

0 means every runner passed every vector. C++, TypeScript, Java, and the CPython extension have no `tessera_seal_runner`: the harness **skips** the TesseraSeal vector directory for those four languages entirely (they consume TesseraID/TesseraHash vectors only) until their v1.4 runners land. There is *no* Python-runner fallback — an earlier revision of this document described one (PQCK-00015), but `run.sh` never implemented it and explicitly documents its absence.

13.3 Hybrid suite `0x06` skip discipline

`tools/conformance/run.sh` filters `hybrid_*.json` vector files per language via `hybrid_supported()`:

```
# Hybrid (suite 0x06) consumed by the four languages with a tessera-seal
# runner: Python (PQCK-00021), Rust (PQCK-00028), Go (PQCK-00029),
# and C (TSEAL-00030). C++/TypeScript/Java/CPython ext have no tessera-seal
# runner, so the harness skips hybrid_*.json (and all seal vectors) for them.
hybrid_supported() {
    case "$1" in
        python) return 0 ;;
        rust)   return 0 ;;
        go)     return 0 ;;
        c)      return 0 ;;
        *)      return 1 ;;
    esac
}
```

`hybrid_supported()` is a property of the *conformance runner*, not of library support. The four languages it returns false for (C++, TypeScript, Java, CPython ext) have no TesseraSeal runner and therefore skip the `hybrid_*.json` files in the harness. At the *library* level, however, C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034) implement suite `0x06` — each pinned by an in-suite test that decodes the frozen `hybrid_*.json` vectors byte-identically — and only Java (TSEAL-00033, backlog) still rejects `0x06` as `UNKNOWN_SUITE` (pinned by its per-language reject test, TSEAL-00044; see section 10). When porting hybrid into the harness for a new language: (1) implement hybrid in the language, (2) update the language's conformance runner to accept 96-byte seeds (section 6), (3) flip the language's `hybrid_supported()` entry, (4) verify the language's PASS count rises by 9 (one per record across the three `hybrid_*.json` files).

13.4 Local invocation

```
bash tools/conformance/run.sh
TESSERA_CONFORMANCE_SKIP="cpython java cpp typescript" bash
    tools/conformance/run.sh
```

Latest state (2026-06-13, Apple M4 Max, alpha quick loop, counts across all three projects — TesseraID + TesseraHash + TesseraSeal; +1 invalid record each since 2026-06-10 from the TSEAL-00049 `KEM_CT_LEN` pin, and C up to 297 with the TSEAL-00030 hybrid port consuming the 9 `hybrid_*.json` records):

Language	PASS	FAIL	Notes
Python	297	0	288 standard + 9 hybrid
C	297	0	288 + 9 hybrid (TSEAL-00030)
Go	297	0	288 + 9 hybrid (PQCK-00029)
Rust	297	0	288 + 9 hybrid (PQCK-00028)

C++, TypeScript, Java, and the CPython extension have no TesseraSeal runner and skip the TesseraSeal vector directory entirely (including `hybrid_*.json`); in the full 8-language run of the same date they consume TesseraID + TesseraHash vectors only (229 PASS each for C++, TypeScript, and Java; 224 PASS + 11 SKIP for the CPython extension).

13.5 CI gate

The GitHub Actions workflow `.github/workflows/conformance.yml` runs on every pull request and push (it is the only automatically-triggered workflow in the repository; the per-language matrices are manual-dispatch), and branch protection on `main` requires its cross-language job. Within that workflow, the 4-language alpha step (Python + C + Go + Rust — the four languages with their own TesseraSeal runners) is the blocking check; the full 8-language step is informational (`continue-on-error`) until the remaining four ports gain TesseraSeal runners.

13.6 Vector generation and the append-only freeze

All vectors are generated against the Python reference (`tessera-seal/python/tessera_seal.py`); the generator script lives at `tessera-seal/conformance/scripts/generate.py`. Shipped vectors are **frozen append-only**: regeneration may append records or add files, but the bytes of an existing committed record never change. The freeze is enforced mechanically by `tools/scripts/check-vector-freeze.sh`.

Full byte-reproducibility of a generator run holds only for the deterministic (seeded-DRBG) vector files. The `hybrid_*.json` emitters draw fresh X25519 ephemerals from the OS CSPRNG at generation time, so re-running the generator does *not* reproduce the committed hybrid files byte-identically. The committed records — whose embedded seeds make *consumption* fully deterministic in every language — are the contract, not the generator run. If a code change makes an existing record fail, that is a spec change: it requires a release-notes entry, appended (never mutated) records, and alignment of every language implementation.

14 Comparison with Existing Formats

A grounded marketplace view as of mid-2026 (this revision: 2026-06-10). TesseraSeal is the only envelope format in the table that offers *both* (a) a pure-PQ option (suites `0x01`/`0x02`) and (b) a key-committing AEAD — with a hybrid alternative (suite `0x06`) layered on the same wire format for callers who want defense-in-depth.

14.1 Envelope formats

Format	Algorithms (2026)	PQ status	Commit	Notes
age v1.2.1 (Valsorda and contributors, 2025)	X25519 + ChaCha20-Poly1305	Experimental plugin only	No	Multi-recipient confusion class still open
Tink v1.17 (Google, 2026)	AES-GCM, AES-GCM-SIV, ChaCha20-Poly, HPKE RFC 9180	ML-KEM tracked, not GA upstream	No	Library, no file-format spec
PASETO v4.local (Arciszewski, 2021)	XChaCha20 + BLAKE2b-MAC (EtM)	None	Yes (by construction)	Inspired our deterministic-nonce + separate-MAC
HPKE-PQ draft 09 (Wood et al., 2026)	X-Wing (X25519 + ML-KEM-768)	Hybrid (KEM 0x647A)	No	The agility-camp answer to the same problem
MLS RFC 9420 (rfc, 2023)	X25519 MLS ciphersuite	Hybrid ML-KEM-768	No	Group messaging, not file
Signal PQXDH (Kret and Schmidt, 2023)	X25519 + Kyber-1024	Hybrid (legacy Kyber)	No	Messaging handshake
Apple iMessage PQ3 (Apple Security Engineering and Architecture, 2024)	ECDH P-256 + Kyber-1024 + rekey	Hybrid (legacy Kyber)	No	Production scale (∼1B devices)
AWS KMS PQ TLS (Amazon Web Services, 2024)	ML-KEM-	Transport hy-	N/A	Not an

The cross-format point: every other hybrid PQ format in 2026 (HPKE-PQ, MLS, Signal, Apple PQ3) is *exclusively* hybrid; every other pure-PQ envelope-shaped artifact is academic. TesseraSeal is the only deployed envelope spec that offers *both* a pure-PQ promise and an opt-in hybrid promise on the same wire, distinguished by a single byte. This dual-promise design draws directly on the immutable-variant-byte discipline that TesseraID (variants `0x01`/`0x02`) and TesseraHash (variant `0x03`) established earlier in the tessera monorepo: a byte *is* a version, never a renegotiation surface.

14.2 KEM cost (Zen 4, cycles; sizes in bytes)

KEM	NIST	Keygen	Encaps	Decaps	pk	sk	ct	Source
ML-KEM-512	1	18 800	22 400	18 200	800	1 632	768	(PQ Code Pac
ML-KEM-768	3	30 100	35 900	29 400	1 184	2 400	1 088	(PQ Code Pac
ML-KEM-1024 (TesseraSeal v1)	**5**	**46 200**	**54 800**	**45 300**	**1 568**	**3 168**	**1 568**	(PQ Code Pac
HQC-128	1	142 000	256 000	384 000	2 249	2 305	4 433	(HQC Submiss
Classic McEliece 348864	1	1.4×10^8	73 000	132 000	261 120	6 492	96	(Classic McEli

14.3 AEAD throughput (1 MiB, single thread, GB/s)

AEAD	Sapphire R.	Zen 4	Apple M3	Notes
AES-256-GCM (TesseraSeal v1)	**10.3**	**9.7**	**6.8**	AES-NI/VAES
AES-256-GCM-SIV	6.1	5.4	3.9	Extra GHASH pass
ChaCha20-Poly1305 (TesseraSeal suite 0x02)	5.8	3.4	3.1	AVX-512 path on x86
AEGIS-128L	28.4	26.1	14.7	RFC 9678 Sep 2025
AEGIS-256	18.9	17.2	10.4	RFC 9678 Sep 2025
Ascon-128a (FIPS 232 final, Aug 2025)	0.55	0.52	0.61	Lightweight, constrained HW

Sources: (Bernstein and Lange, 2025; Intel Corporation, 2025; rfc, 2025). AEGIS is 2–3× faster than AES-256-GCM. It was evaluated for v1.2 at suite `0x03` (PQCK-00020) and *deferred*: as of the v1.2 evaluation (2026-05-21) verified AEGIS implementations are not yet polyglot-ready across Python, Go, Java, and OpenSSL. Revisit when verified implementations land.

14.4 Reference benchmarks (Apple M4 Max, ops/sec, v1.3)

End-to-end TesseraSeal operations as measured by the per-language bench harnesses on Apple M4 Max (single P-core, release builds). Numbers are ops/sec; higher is better. "Hybrid" rows refer to suite `0x06` (section 6).

Operation	Python	C	Go	Rust
keygen (ML-KEM-1024)	398	24 921	24 666	11 530
keygen (hybrid `0x06`)	388	— v1.4	17 521	8 073
encrypt 1 KiB (AES-256-GCM)	342	20 966	20 788	9 372
encrypt 1 KiB (hybrid `0x06`)	305	— v1.4	10 000	4 653
decrypt 1 KiB (AES-256-GCM)	259	8 689	20 232	17 686
decrypt 1 KiB (hybrid `0x06`)	248	— v1.4	13 724	11 485
encrypt 1 MiB (AES-256-GCM)	232	447	685	131
decrypt 1 MiB (AES-256-GCM)	202	405	701	135

Read: Go's TesseraSeal port is the fastest because Go 1.26 stdlib `crypto/aes` routes through ARMv8 AES instructions on Apple Silicon automatically and the GC compiler inlines the ML-KEM hot path tightly. C trails Go on `decrypt 1 KiB` because the OpenSSL `EVP_KEM Decaps` path performs an additional dispatch step. Rust's 1 MiB throughput is regressed because `aes-gcm 0.10` does not enable ARMv8 AES intrinsics on stable Rust without a nightly `target_feature` cfg; this is tracked as PQCK-00027 (deferred to v1.4) and is expected to recover to C-class throughput once `aes-gcm 0.11` ships. Hybrid encrypt is roughly 2× slower than the AES-only path because each envelope adds one X25519 ephemeral keygen and one DH; hybrid decrypt adds only one DH.

14.5 Deployment scale (mid-2026)

- Cloudflare Radar (Apr 2026): ∼38% of outbound TLS 1.3 to origins uses `X25519MLKEM768`; ∼32% of inbound from browsers (Cloudflare, 2026).
- Chrome: `X25519MLKEM768` default-on Chrome 131 (Nov 2024); `X25519Kyber768` fully removed in Chrome 141.
- OpenSSH 10.0 (Apr 2025): `mlkem768x25519-sha256` default key exchange.
- Apple iMessage PQ3: iOS 17.4 (Mar 2024), ∼1B devices.
- Signal PQXDH: mandatory client ≥ Sep 2023.
- AWS KMS PQ TLS to endpoints: GA Oct 2024.
- GCP Cloud KMS: PQ signing GA Feb 2026, PQ KEM preview.

The ecosystem reality: hybrid (X-Wing or TLS-style concat) has won the transport-protocol fight. TesseraSeal is deliberately a counter-example for the envelope-format niche where unambiguous "PQ-only" is a feature, not a bug.

15 Appendices

A. Residual-risk acknowledgment for the pure-PQ core promise

TesseraSeal's core promise (suites `0x01` and `0x02`) is post-quantum-only. If ML-KEM-1024 is later structurally broken (a sub-exponential Module-LWE attack), every `0x01`/`0x02` envelope loses confidentiality with no classical fallback. This is a 30–50 year risk accepted in exchange for the unambiguous PQ-only promise. The mitigation path is now operational rather than aspirational:

1. Suite byte `0x06` (PQCK-00021, v1.2; section 6) is the opt-in hybrid suite (X25519 + ML-KEM-1024 + AES-256-GCM + UtC). Callers who want belt-and-suspenders coverage select `0x06` at `encrypt` time. The wire byte at offset 7 records which promise the envelope makes.
2. Long-lived ciphertexts written under suites `0x01`/`0x02` can be re-encrypted under `0x06` (or a future suite) within a defined window. TesseraSeal v1.x is not an archive format; long-lived storage should plan re-encryption.
3. Suite `0x01`/`0x02` decoders remain available indefinitely (suite bytes are immutable post-allocation, matching the variant-byte discipline of TesseraID and TesseraHash).

We do *not* mitigate by hedging suites `0x01`/`0x02` themselves. The pure-PQ promise of those suites is preserved verbatim from v1.0; the hybrid escape valve sits in a separate suite-byte allocation. This is the design choice that lets the same wire format carry two distinct guarantees without diluting either.

B. Variant byte allocation across tessera

Byte	Owner	Meaning	Status
`0x00`	—	Permanently reserved	MUST reject
`0x01`	TesseraID	Standard generator	assigned
`0x02`	TesseraID	HLC generator	assigned
`0x03`	TesseraHash	SHAKE-256 / 64 B tail	assigned
`0x04`	TesseraSeal	Encryption envelope v1	assigned (this document)
`0x05`–`0xFE`	—	Reserved-future	MUST reject
`0xFF`	—	Permanently reserved	MUST reject

Cross-rejection is normative: TesseraID and TesseraHash decoders MUST reject `0x04`; TesseraSeal decoders MUST reject `0x01`, `0x02`, `0x03`, and every byte in `0x05`–`0xFF` in v1. Each project's `invalid.json` conformance file pins the cross-rejection.

C. Suite byte allocation

Suite	Algorithms	Status
`0x01`	ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.0 mandatory (pure PQ)
`0x02`	ML-KEM-1024 + ChaCha20-Poly1305 + UtC commit + HKDF-SHA3-256	v1.0 optional (pure PQ)
`0x06`	X25519 + ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.2 optional (hybrid)
`0x07`	RESERVED: ML-KEM-1024+HQC hybrid or HQC-only (section 4)	reserved; MUST reject
`0x00`, `0x03`–`0x05`, `0x08`–`0xFF`	Reserved-future	MUST reject

Earmarked future suites (not normative in v1.x):

- `0x03`: ML-KEM-1024 + AEGIS-128L + UtC commit. Evaluated and deferred in PQCK-00020 (v1.2, 2026-05-21): AEGIS libraries not yet polyglot-ready. The byte remains reserved-future; revisit when libcrux, OpenSSL, and Go stdlib all ship verified AEGIS implementations.
- `0x07`: **explicitly RESERVED** (TSEAL-00048) for the HQC contingency — an ML-KEM-1024+HQC hybrid or HQC-only successor suite (National Institute of Standards and Technology, 2025); allocated upon NIST HQC FIPS finalization (draft slipped; final ∼2027); activation trigger = Module-LWE confidence loss or HQC standardization. See section 4 for the full roadmap paragraph.
- Hardened-implementation variants (masked ML-KEM, side-channel-protected X25519) would each take a new suite byte rather than modify `0x01`/`0x02`/`0x06`.

D. Test-vector schema

Each KAT file is a JSON array of records. Common fields:

```
{
  "name":          "human-readable id, unique within the file",
  "op":            "encrypt | decrypt | peek_header | commit | ...",
  "input_hex":     "raw bytes for decode-from-binary ops",
```

```
  "input":        { ... op-specific structured input },
  "expected":     { ... op-specific structured output },
  "should_fail":  false,
  "error_kind":   "DECRYPT_FAILED | UNKNOWN_VARIANT | UNKNOWN_SUITE |
      INVALID_LEN"
}
```

Full schema in `tessera-seal/conformance/README.md`.

E. Acknowledgments

TesseraID and TesseraHash for the polyglot-spec-first discipline this whitepaper inherits. Bellare and Hoang for the UtC construction (Bellare and Hoang, 2022). The OpenSSL maintainers for the ML-KEM-1024 `EVP_KEM` integration shipped in OpenSSL 3.5; the RustCrypto `ml-kem` authors for the pinned Rust source; the Go core team for landing `crypto/mlkem` in the 1.26 stdlib; the `kyber-py` maintainers (NCC Group audit, Q2 2024); and the Cryspen team for `mlkem-native` (Cryspen and PQ Code Package, 2025), evaluated during the v1.0 drafts (the shipped C reference uses OpenSSL instead; section 5). The X25519 sources used by hybrid suite `0x06`: `x25519-dalek` v2 (Rust), Go stdlib `crypto/ecdh`, and `cryptography` (Python). The X-Wing draft authors (Connolly et al., 2025) for the combiner design idiom. The KyberSlash (Bernstein et al., 2024), PQShield (PQShield, 2024), SPA-CCA (Guo et al., 2024), and GoFetch (Chen et al., 2024) disclosure teams, whose findings shape the side-channel disclaimer.

F. Release timeline (v1.0 – v1.3.x)

Release	Date	Tickets	Headline
v1.0 alpha	2026-05-20	PQCK-00001–00010	Spec + Python/C/Go/Rust references + plugin protocol + cross-language conformance harness. Suites `0x01`/`0x02` only.
v1.1	2026-05-21	PQCK-00011–00019	C++ / TypeScript / Java / CPython ext ports; `.tseal-key-v1.1` keyfile format (embeds pubkey); CSPRNG path rewire; AEGIS-128L evaluation (deferred). C switched from `mlkem-native` to OpenSSL 3.5 `EVP_KEM`; Java switched from BouncyCastle 1.79+ to SunJCE JEP 452; Go switched from `filippo.io/mlkem` to stdlib `crypto/mlkem`.
v1.2	2026-05-21	PQCK-00020–00026	**Hybrid suite `0x06`** (Python only); reference CLI binary; GitHub Actions matrix; bench sweep; internal security audit doc; AEGIS deferral recorded.
v1.3 partial	2026-05-21	PQCK-00027–00029	Hybrid suite `0x06` ports to Rust (PQCK-00028) and Go (PQCK-00029). AES-NI/ARMv8 intrinsics deferral for Rust (PQCK-00027).
v1.3 hardening	2026-06-08–10	TSEAL-00035–00045	BadKey suite/key consistency completed across all eight ports (TSEAL-00035–00038); CLI suite-flag hard-error (TSEAL-00039); keyfile KDF block, fail-closed (TSEAL-00040); KAT expansion + append-only vector freeze; suite-`0x06` rejection tests in the five non-hybrid ports (TSEAL-00044); whitepaper truthfulness errata (TSEAL-00045, this revision).

v1.4 backlog (not normative in this document): the hybrid suite `0x06` port to C landed (TSEAL-00030, 2026-06-13 — a 4-way byte-identical KAT contract with Python/Rust/Go), and library-level hybrid then landed in C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034) on the same day, each verified by an in-suite test that decodes the frozen `hybrid_*.json` vectors byte-identically without a tessera-seal conformance runner; Java (TSEAL-00033) is the sole remaining hybrid port, pending a JDK 24+ build/verify environment. Also outstanding: adding tessera-seal conformance runners for C++, TypeScript, Java, and the CPython extension so their (already library-level) coverage is harness-enforced; and reopening the Rust AES-NI/ARMv8 intrinsics work deferred in PQCK-00027. The automated CI timing-histogram regression (`security-audit-v1.2.md` open item #1) landed in TSEAL-00048 (an earlier revision listed it as PQCK-00035, a ticket number since consumed by the BadKey suite/key-consistency work released 2026-06-08). Ticket-prefix note: PQCK-*NNNNN* identifiers are historical (pre-rename); the same numbering continues as TSEAL-*NNNNN*, and `tessera-seal/CLAUDE.md` is the authoritative ticket registry where this document and it disagree.

G. Author and document provenance

Author: Bahram Samani, `tessera` monorepo maintainer.

This document is built from LaTeX sources under `tessera-seal/latex/` (`main.tex` + `secNN_*.tex`); the build chain is `pdflatex` → `bibtex` → `pdflatex` × 2 per the Makefile. The artifact at `tessera-seal/latex/TesseraSeal_Whitepaper.pdf` is regenerated and committed alongside any normative change.

Authority order (when sources disagree): this whitepaper, then the Python reference at `tessera-seal/python/tes` then the canonical C ABI at `tessera-seal/c/tessera_seal.h`, then every other language. A cross-language disagreement is never resolved by changing the spec to match an outlier; the outlier is fixed.

Threat-model checklist: the internal pre-external-audit walk-through is at `tessera-seal/docs/security-audit` (PQCK-00025). It enumerated six open items for an external auditor; of these, the CI timing histogram (automated in TSEAL-00048) and the hybrid C port (TSEAL-00030, 2026-06-13) have since landed, leaving four (FIPS 140-3 path, synthetic-key timing-pad invariant testing, plugin protocol fuzz harness, memory-zeroization audit).

Side-channel posture: this document and the companion `tessera-seal/docs/side-channel-disclaimer.md` are normative. v1.x mitigates software-timing side channels only; not power, EM, frequency-scaling, or prefetcher attacks.

Conformance gate: `tools/conformance/run.sh` backs the CI gate — `.github/workflows/conformance.yml` runs on every pull request and push, and branch protection on `main` requires its cross-language job (the 4-language alpha step blocks; the full 8-language step is informational). Latest run (2026-06-13, Apple M4 Max alpha quick loop): 1188 PASS, 0 FAIL across TesseraID + TesseraHash + TesseraSeal, four alpha languages ($297 + 297 + 297 + 297$ — C reached parity once its hybrid `0x06` port landed, TSEAL-00030; section 13).

References

SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, Aug. 2015. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Argon2 memory-hard function for password hashing and proof-of-work applications, 2021. URL https://datatracker.ietf.org/doc/html/rfc9106.

The messaging layer security (MLS) protocol, 2023. URL https://datatracker.ietf.org/doc/html/rfc9420.

Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS PUB 203, National Institute of Standards and Technology, Aug. 2024. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

The AEGIS family of authenticated encryption algorithms, Sept. 2025. URL https://www.rfc-editor.org/rfc/rfc9678.

A. Albertini, T. Duong, S. Gueron, S. Kölbl, A. Luykx, and S. Schmieg. How to abuse and fix authenticated encryption without key commitment. IACR ePrint 2020/1456, 2020. URL https://eprint.iacr.org/2020/1456.

Amazon Web Services. Using post-quantum TLS with AWS KMS, 2024. URL https://docs.aws.amazon.com/kms/latest/developerguide/pqtls.html.

Apple Security Engineering and Architecture. imessage with PQ3: A new state of the art in quantum-secure messaging at scale, 2024. URL https://security.apple.com/blog/imessage-pq3/.

S. Arciszewski. PASETO v4.local specification, 2021. URL https://github.com/paseto-standard/paseto-spec/blob/master/docs/01-Protocol-Versions/Version4.md.

F. Bäumer, M. Brinkmann, and J. Schwenk. Terrapin Attack: Breaking SSH channel integrity by sequence-number manipulation (CVE-2023-48795). USENIX Security 2024, 2023. URL https://terrapin-attack.com/.

M. Bellare and V. T. Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2022/268. IACR ePrint 2022/268.

M. Bellare and V. T. Hoang. Succinctly-committing authenticated encryption. CRYPTO 2024; IACR ePrint 2024/875, 2024. URL https://eprint.iacr.org/2024/875.

D. J. Bernstein and T. Lange. eBACS: ECRYPT benchmarking of cryptographic systems (SUPERCOP), 2025. URL https://bench.cr.yp.to/.

D. J. Bernstein et al. KyberSlash 1 & 2: Timing-based key recovery on ML-KEM, 2024. URL https://kyberslash.cr.yp.to/.

P. Bose, V. T. Hoang, and S. Tessaro. Revisiting AES-GCM-SIV: Multi-user security, faster key derivation, and better bounds. IACR ePrint 2017/702, 2018. URL https://eprint.iacr.org/2017/702.

C2SP. The age plugin protocol, 2023. URL https://github.com/C2SP/C2SP/blob/main/age-plugin.md.

J. Chan and P. Rogaway. On committing authenticated encryption. IACR ePrint 2022/1260 (CTX construction), 2022. URL https://eprint.iacr.org/2022/1260.

B. Chen et al. GoFetch: Breaking constant-time cryptographic implementations using data memory-dependent prefetchers. USENIX Security 2024, 2024. URL https://gofetch.fail/.

Classic McEliece Submission Team. Classic McEliece: Implementation notes, 2022. URL https://classic.mceliece.org/mceliece-impl-20221023.pdf.

Cloudflare. Radar: Post-quantum TLS adoption, 2026. URL https://radar.cloudflare.com/adoption-and-usage.

D. Connolly, P. Schwabe, and B. Westerbaan. X-Wing: General-purpose hybrid post-quantum KEM. draft-connolly-cfrg-xwing-kem, IETF CFRG, 2025. URL https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/.

Cryspen and PQ Code Package. mlkem-native: Formally verified ML-KEM for C/AArch64/AVX2, 2025. URL https://github.com/pq-code-package/mlkem-native.

J. Don, S. Fehr, C. Majenz, and C. Schaffner. Online-extractability in the quantum random-oracle model. In *EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2021/280.

Google. Tink releases, 2026. URL https://github.com/tink-crypto/tink/releases.

Google Cloud. Post-quantum cryptography in Cloud KMS, 2026. URL https://cloud.google.com/kms/docs/post-quantum-cryptography.

P. Grubbs, J. Lu, and T. Ristenpart. Message franking via committing authenticated encryption. In *CRYPTO 2017*, 2017. URL https://eprint.iacr.org/2017/664.

Q. Guo, T. Johansson, and B.-Y. Yang. Single-trace SPA-CCA on unmasked ML-KEM on Cortex-M4. IACR ePrint 2024/2051, 2024. URL https://eprint.iacr.org/2024/2051.

D. Hofheinz, K. Hövelmanns, and E. Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *TCC 2017*, 2017. URL https://eprint.iacr.org/2017/604.

HQC Submission Team. HQC performance, 2025. URL https://pqc-hqc.org/doc/hqc-performance.pdf.

Intel Corporation. Data operand independent timing instructions (DOIT), 2022. URL https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/data-operand-independent-timing-isa-guidance.html.

Intel Corporation. Intel ISA-L: Intelligent storage acceleration library, 2025. URL https://github.com/intel/isa-l_crypto.

E. Kret and R. Schmidt. The PQXDH key agreement protocol, 2023. URL https://signal.org/docs/specifications/pqxdh/.

A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, IETF, 2016. URL https://www.rfc-editor.org/rfc/rfc7748.

J. Len, P. Grubbs, and T. Ristenpart. Partitioning oracle attacks. USENIX Security 2021; IACR ePrint 2020/1491, 2021. URL https://eprint.iacr.org/2020/1491.

National Institute of Standards and Technology. NIST selects HQC as fifth algorithm for post-quantum encryption. News announcement, March 11, 2025; draft standard expected 2026, final expected 2027, 2025. URL https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption.

PQ Code Package. mlkem-native benchmarks, 2026. URL https://github.com/pq-code-package/mlkem-native#benchmarks.

PQShield. Plugging timing leaks in Kyber/ML-KEM via masking, 2024. URL https://pqshield.com/blog/power-side-channel-ml-kem.

F. Valsorda and contributors. age releases (filippo.io/age), 2025. URL https://github.com/FiloSottile/age/releases.

Y. Wang, R. Paccagnella, E. T. He, H. Shacham, C. W. Fletcher, and D. Kohlbrenner. Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86. USENIX Security 2022, 2022. URL https://www.hertzbleed.com/.

C. A. Wood et al. Hybrid HPKE for post-quantum KEMs (draft-ietf-hpke-pq-09), 2026. URL https://datatracker.ietf.org/doc/draft-ietf-hpke-pq/.